

OOP: concepts 4

Advanced: Practice 1  

In the object-oriented programming paradigm, code is organised into classes. Within a class you will find attributes and methods with access modifiers that specify which of these properties can be accessed from outside the class.

Study the following class definition:

Pseudocode

```

1 CLASS Elf
2     PRIVATE strength: Integer
3     PRIVATE speed: Integer
4     PUBLIC power: String
5
6     PUBLIC PROCEDURE Elf(given_strength, given_speed)
7         strength = given_strength
8         speed = given_speed
9         power = "Archery"
10    ENDPROCEDURE
11
12    PUBLIC FUNCTION get_strength()
13        RETURN strength
14    ENDFUNCTION
15 ENDClass
16
17 aegnor = NEW Elf(20, 50)
    
```

The table below gives a list of terms that are relevant to OOP classes. Use the class definition above to pick an appropriate example for each of the terms. Drag the example into the cell next to the term.

Term	Label
An attribute	
A method	
A class	
A reference variable	
An access modifier	
A data type	
A parameter	

Items:

strengthIntegerPRIVATEElfægnorget_strengthgiven_strength

Quiz:
STEM SMART Computer Science Week 21

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



Constructor method definition

Advanced: Practice 1  

What is a constructor method?

- ☐ A method that is used to instantiate an object.
- ☐ A method that is used to create a child class from the definition of a parent class
- ☐ A public method that is used to set the values of private attributes.
- ☐ A method that is used to get the values of public attributes outside the class in which they were defined

Quiz:

STEM SMART Computer Science Week 21

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



Encapsulation

Advanced: Practice 1  

Select one statement that describes why the principle of **encapsulation** is important for the design of OOP programs.

- ☐ It ensures that child classes can take the attributes and methods of the parent class.
- ☐ It allows the methods of child classes to behave in different ways to those inherited from a parent class.
- ☐ It enables the creation of objects with specific states and behaviour.
- ☐ It ensures that any interaction with an object, specifically the manipulation of its data, is only allowed via its public interface.

Quiz:

[STEM SMART Computer Science Week 21](#)

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

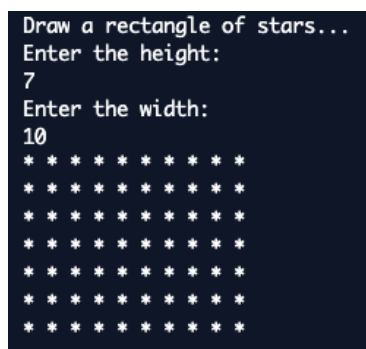


OOP: complete code 5

Advanced: Practice 1  

Steve is learning object-oriented programming. He wants to write some code that will draw a simple rectangle, so he has created a class called `Rectangle`, with two attributes: `height` and `width` and a method `draw`.

The `draw` method will display a rectangle of stars of a given height and width. For example, a rectangle of height 7 and width 10, will look like this:



```
Draw a rectangle of stars...
Enter the height:
7
Enter the width:
10
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Example output from Steve's program

Pseudocode

```
1 CLASS Rectangle
2     PRIVATE width
3     PRIVATE height
4
5     PUBLIC PROCEDURE Rectangle(given_width, given_height)
6         width = given_width
7         height = given_height
8     ENDPROCEDURE
9
10    PUBLIC PROCEDURE draw()
11        FOR row = 0 TO height
12            FOR column = 0 TO width
13                PRINT("* ")
14            NEXT column
15            PRINT() # Print new line.
16        NEXT row
17    ENDPROCEDURE
18 ENDCLASS
19
20 PRINT("Draw a rectangle of stars...")
21 input_height = INPUT("Enter the height: ")
22 input_width = INPUT("Enter the width: ")
23
24 my_shape = NEW Rectangle(_____, _____) // Missing code
25 my_shape.draw()
```

In the pseudocode shown above, a new Rectangle object is instantiated and its **draw** method is called. However, the line of code that instantiates the object has missing arguments. Can you identify what they should be?

Enter your answer as **argument1**, **argument2** using a comma to separate the arguments.

Quiz:

STEM SMART Computer Science Week 21

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



OOP: complete code 6

Advanced: Practice 1  

The following class has been defined using pseudocode.

Pseudocode

```
1 CLASS Radio
2     PRIVATE volume: integer
3     PRIVATE station: string
4     PRIVATE on: Boolean
5
6     PUBLIC PROCEDURE Radio(given_station)
7         station = given_station
8         volume = 3
9         on = False
10    ENDPROCEDURE
11
12    PUBLIC FUNCTION get_volume()
13        RETURN volume
14    ENDFUNCTION
15
16    PUBLIC FUNCTION get_station()
17        RETURN station
18    ENDFUNCTION
19
20    PUBLIC FUNCTION is_on()
21        RETURN on
22    ENDFUNCTION
23
24    PUBLIC PROCEDURE set_volume(new_volume)
25        volume = new_volume
26    ENDPROCEDURE
27
28    PUBLIC PROCEDURE set_station(new_station)
29        station = new_station
30    ENDPROCEDURE
31
32    PUBLIC PROCEDURE switch()
33        IF on == True THEN
34            on = False
35        ELSE
36            on = True
37        ENDIF
38    ENDPROCEDURE
39 ENDClass
```

Which of the following options (expressed in pseudocode) will correctly create an instance of a radio object?

- ☐ `radio1 = NEW Radio("Capital FM", 3, True)`
- ☐ `Radio = NEW radio1("Capital FM", 3, True)`
- ☐ `radio1 = NEW Radio("Capital FM")`
- ☐ `Radio = NEW radio1("Capital FM")`

Quiz:

STEM SMART Computer Science Week 21

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



OOP: complete code 7

Advanced: Challenge 2  

The following class has been defined using pseudocode.

Pseudocode

```
1 CLASS Radio
2     PRIVATE volume: integer
3     PRIVATE station: string
4     PRIVATE on: Boolean
5
6     PUBLIC PROCEDURE Radio(given_station)
7         station = given_station
8         volume = 3
9         on = False
10    ENDPROCEDURE
11
12    PUBLIC FUNCTION get_volume()
13        RETURN volume
14    ENDFUNCTION
15
16    PUBLIC FUNCTION get_station()
17        RETURN station
18    ENDFUNCTION
19
20    PUBLIC FUNCTION is_on()
21        RETURN on
22    ENDFUNCTION
23
24    PUBLIC PROCEDURE set_volume(new_volume)
25        volume = new_volume
26    ENDPROCEDURE
27
28    PUBLIC PROCEDURE set_station(new_station)
29        station = new_station
30    ENDPROCEDURE
31
32    PUBLIC PROCEDURE switch()
33        IF on == True THEN
34            on = False
35        ELSE
36            on = True
37        ENDIF
38    ENDPROCEDURE
39
40 ENDClass
```

An object has been created based on the class definition shown above and assigned to a reference variable called `my_radio`. Which of the following options (expressed in pseudocode) will set the volume of this radio object to 6?

- ☐ `my_radio.set_volume(6)`
 - ☐ `my_radio.set_volume() = 6`
 - ☐ `my_radio.volume = 6`
-
-
-

Quiz:

STEM SMART Computer Science Week 21

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



OOP: complete code 2

Advanced: Practice 1  

Jemma is developing a game for her younger brother that takes place in space. So far, she has created part of the definition of the **Planet** class.

Select the statement that can create an object of the **Planet** class called **mars** so that it is habitable, has two satellites, is 227.9 million kilometres away from the sun, and has a dominant colour of red.

Pseudocode

```
1 CLASS Planet
2     PRIVATE distance_sun // Float given in million km
3     PRIVATE satellites // Integer
4     PRIVATE habitable // Boolean
5     PRIVATE main_colour // String
6
7     PUBLIC PROCEDURE Planet(given_distance_sun, given_satellites, given_habitable, given_main_colour)
8         distance_sun = given_distance_sun
9         satellites = given_satellites
10        habitable = given_habitable
11        main_colour = given_main_colour
12    ENDPROCEDURE
13
14    PROCEDURE set_distance_sun(number)
15        distance_sun = number
16    ENDPROCEDURE
17 ENDClass
```

- ☐ mars = NEW Planet(227.9 million km, 2, True, red)
- ☐ mars = NEW Planet(227.9, 2, True, 'red')
- ☐ mars = NEW Planet(mars.set_distance_sun(227.9), 2, True, 'red')
- ☐ Planet = NEW mars(True, 2, 227.9, red)

Quiz:

STEM SMART Computer Science Week 21

OOP: complete code 3

Advanced: Practice 2 

Jemma has extended a game that she is developing for her younger brother that takes place in space. A description of the **Planet** class that she has defined in her program is shown in **Figure 1**.

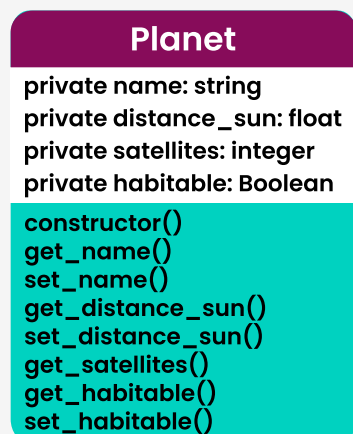


Figure 1: Planet class

The **Galaxy** class contains a **planets** array that stores the objects of type **Planet** that are used in the game.

It also contains a **count_habitable_planets** method that searches the **planets** array and shows the number of planets that are habitable.

Pseudocode

```
1 CLASS Galaxy
2     PRIVATE Planet planets[15] // Array of type Planet
3
4     PUBLIC FUNCTION count_habitable_planets()
5         total_habitable = 0 // Number of habitable planets
6         FOR count = 0 TO LEN(planets) - 1
7             IF ..... THEN // Missing code
8                 total_habitable = total_habitable + 1
9         ENDIF
10    NEXT count
11    RETURN total_habitable
12
13
```

```
ENDFUNCTION  
ENDCLASS
```

Part A

Complete the statement for the missing code in the **IF** statement of the **count_habitable_planets** method:

IF . == True THEN

Items:

Galaxy[count] planets Planet get_habitable() planets[count]
 habitable

Part B

Jemma has made a new instance of the **Galaxy** class and assigned it to the reference variable **milky_way**.

She has provided the **milky_way** object with test data for the array of **Planets** so they she can check whether the **count_habitable_planets** method works.

Using the statements below, call the **count_habitable_planets** method and output the results from the variable **habitable_planets**.

```
habitable_planets = .  
PRINT()
```

Items:

Planet count_habitable_planets() habitable planets milky_way
 Galaxy habitable_planets total_habitable

OOP: complete code 1

Advanced: Challenge 2 

Mike is creating a game inspired by *The Lord of the Rings*. Each character of the game can belong to one of the following tribes: Elves, Dwarves, Hobbits, Men, Wizards, Orcs, and Trolls. The characters that belong to the tribe of Elves have a commonly known name and a secret elf name.

A part of the definitions of the **Character** and **Elf** classes is presented below. **SUPER** is used to call the constructor of the **Character** parent class from the **Elf** child class.

Pseudocode

```
1 CLASS Character
2     PRIVATE strength: Integer
3     PRIVATE speed: Integer
4     PUBLIC power: String
5     PUBLIC name: String
6
7     PUBLIC PROCEDURE Character(given_strength, given_speed, given_name)
8         strength = given_strength
9         speed = given_speed
10        power = "None"
11        name = given_name
12    ENDPROCEDURE
13
14    PUBLIC FUNCTION get_strength()
15        RETURN strength
16    ENDFUNCTION
17 ENDClass
18
19 CLASS Elf EXTENDS Character
20     PRIVATE elf_name: String
21
22     PUBLIC PROCEDURE Elf(given_strength, given_speed, given_elf_name, given_name)
23         SUPER(given_strength, given_speed, given_name)
24         elf_name = given_elf_name
25     ENDPROCEDURE
26
27     PUBLIC FUNCTION get_elf_name()
28         RETURN elf_name
29     ENDFUNCTION
30 ENDClass
```

Part A

Enter the name (identifier) of **one** of the attributes. Do not include anything else.

Part B

Mike wants to create an instance of the **Elf** class and assign it to the reference variable **my_elf**.

Complete the statement below for creating a new **Elf** object that has the name "Legolas" and the secret elf name "Greenleaf".

my_elf = NEW (200, 1000, ,)

Items:

- Elf
- given_elf_name
- "Greenleaf"
- Character
- "Legolas"
- given_name

Part C

An instance of the `Elf` class called `my_elf` is created, and then an output (`print`) statement is used to demonstrate the value of an attribute.

Pseudocode

```
1  PROCEDURE new_elf()
2      my_elf = NEW Elf(200, 1000, "Greenleaf", "Legolas")
3      PRINT(.....) // Missing code for the output statement
4  ENDPROCEDURE
```

Look at the list of output statements and select the **two** statements that will cause an error if they are used to display the value of an attribute.

- ☐ `PRINT(my_elf.get_elf_name())`
 - ☐ `PRINT(my_elf.power)`
 - ☐ `PRINT(my_elf.name)`
 - ☐ `PRINT(my_elf.get_strength())`
 - ☐ `PRINT(my_elf.elf_name)`
 - ☐ `PRINT(my_elf.speed)`
-
-

Quiz:

STEM SMART Computer Science Week 21

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.



OOP: complete code 4

Advanced: Challenge 1

Maxime is programming a horse riding simulation where a rider travels to various historic castles in the UK with different breeds of horses. Each breed of horse can travel at a different speed, for example, a **Thoroughbred** can travel at **88 km/h**, a **QuarterHorse** at **70 km/h**, and an **Appaloosa** at **65 km/h**. She has created three classes for her program.

Castle

private distance: float
private barriers: Boolean

constructor(given_distance, given_barriers)
get_distance()
set_distance()
get_barriers()
set_barriers()

Horse

private speed: integer

constructor(given_speed)
get_speed()
set_speed()

Castle and Horse classes

Rider

private skill: integer

constructor(given_skill)
get_skill()
set_skill()
travel_time(target_castle, current_horse)

Rider class

The **travel_time** method of the **Rider** class has the following functionality:

- If the **skill** of the person travelling is greater than 8 and there are no **barriers** on the way to the castle, then the **time** required to reach the castle is calculated as the **distance** to the castle divided by the **speed** of the horse
- In any other scenario, the time taken to reach the castle is **doubled**.

Part A

Enter the statement that completes the code under the **IF** statement.

Pseudocode

```
1 PUBLIC FUNCTION travel_time(target_castle, current_horse)
2     IF skill > 8 AND target_castle.get_barriers() == False
3         time_required = ..... // Missing code
4     ELSE
5         // It takes double the time
6     ENDIF
7
8     RETURN time_required
9 ENDFUNCTION
```

Part B

Based on the implementation of the `travel_time` method, what is the time required to reach the castle if the below set of objects are used in the simulation? Give your answer as a number.

Pseudocode

```
1  my_castle = new Castle(130, True)
2
3  my_appaloosa = new Horse(65)
4
5  my_rider = new Rider(9)
```

Quiz:

STEM SMART Computer Science Week 21

All teaching materials on this site are available under a CC BY-NC-SA 4.0 license, except where otherwise stated.

